# All crashes in one

**Create the File :**

```
nano all_crashes.c
```

**Paste this code in the file  :**

```
#include <stdio.h>
#include <stdlib.h>

/*
Crash Type 1: NULL Pointer Dereference
- Pointer explicitly set to NULL (0x0)
- Writing causes immediate SIGSEGV
*/
void null_deref() {
    int *p = NULL;
    *p = 1;
}

/*
Crash Type 2: Wild Pointer
- Pointer is uninitialized (contains garbage address)
- May crash or corrupt random memory
*/
void wild_pointer() {
    int *p;
    *p = 1;
}

/*
Crash Type 3: Heap Overflow
- Allocated space for 3 integers
- Writing far beyond boundary (index 10)
- Corrupts heap memory
*/
```

```
void heap_overflow() {
    int *a = malloc(3 * sizeof(int));
    a[10] = 1;   // out-of-bounds write
    free(a);
}

/*
Crash Type 4: Use-After-Free
- Memory is freed but pointer still used
- Undefined behavior (dangerous in real systems)
*/
void use_after_free() {
    int *p = malloc(sizeof(int));
    free(p);
    *p = 1;   // accessing freed memory
}

/*
Crash Type 5: Stack Overflow
- Infinite recursion
- Stack keeps growing until limit exceeded
*/
void stack_overflow() {
    stack_overflow();
}

int main() {
    int choice;

    printf("\nChoose crash type:\n");
    printf("1. NULL Dereference\n");
    printf("2. Wild Pointer\n");
    printf("3. Heap Overflow\n");
    printf("4. Use After Free\n");
    printf("5. Stack Overflow\n");
    printf("Enter choice: ");

    scanf("%d", &choice);
```

```
    if (choice == 1) null_deref();
    else if (choice == 2) wild_pointer();
    else if (choice == 3) heap_overflow();
    else if (choice == 4) use_after_free();
    else if (choice == 5) stack_overflow();
    else printf("Invalid choice\n");

    return 0;
}
```

**SAVE :**

```
CTRL + O → Enter → CTRL + X
```

**Compile with debugging :**

```
gcc -g all_crashes.c -o all_crashes
```

**Run Normally :**

```
./all_crashes
```

## Trying different inputs (1-5) :

`Choice → 1 :`

**Crash Type** : NULL deref

**Behavior** : Immediate crash.

```
-bash: ./all: No such file or directory
dharmin@LAPTOP-DE5D0E9H:~$ ./all_crashes

Choose crash type:
1. NULL Dereference
2. Wild Pointer
3. Heap Overflow
4. Use After Free
5. Stack Overflow
Enter choice: 1
Segmentation fault (core dumped)
dharmin@LAPTOP-DE5D0E9H:~$
```

`Choice → 2 :`

**Crash Type : Wild pointer**

**Behavior : Crash OR weird behavior**

```
dharmin@LAPTOP-DE5D0E9H:~$ ./all_crashes

Choose crash type:
1. NULL Dereference
2. Wild Pointer
3. Heap Overflow
4. Use After Free
5. Stack Overflow
Enter choice: 2
Segmentation fault (core dumped)
```

`Choice → 3 :`

**Crash Type : Heap overflow**

**Behavior : May crash later.**

```
dharmin@LAPTOP-DE5D0E9H:~$ ./all_crashes

Choose crash type:
1. NULL Dereference
2. Wild Pointer
3. Heap Overflow
4. Use After Free
5. Stack Overflow
Enter choice: 3
```

No error message .

`Choice → 4 :`

**Crash Type : Use-after-free**

**Behavior : Unpredictable**

```
dharmin@LAPTOP-DE5D0E9H:~$ ./all_crashes

Choose crash type:
1. NULL Dereference
2. Wild Pointer
3. Heap Overflow
4. Use After Free
5. Stack Overflow
Enter choice: 4
```

No error message .

`Choice → 5 :`

**Crash Type : Stack Overflow**

**Behavior : Crash after recursion.**

```
dharmin@LAPTOP-DE5D0E9H:~$ ./all_crashes

Choose crash type:
1. NULL Dereference
2. Wild Pointer
3. Heap Overflow
4. Use After Free
5. Stack Overflow
Enter choice: 5
Segmentation fault (core dumped)
```

# Debugging with GDB :

```
gdb ./all_crashes
```

## Run Inside GDB :

```
run
```

## Trying different inputs (1-5) in GDB :

`Choice → 1(GDB) :`

```
Choose crash type:
1. NULL Dereference
2. Wild Pointer
3. Heap Overflow
4. Use After Free
5. Stack Overflow
Enter choice: 1

Program received signal SIGSEGV, Segmentation fault.
0x00005555555551fd in null_deref () at all_crashes.c:12
12      *p=1;
```

### Back trace :

```
bt
```

Shows exact crash path .

### Inspecting path using `bt` for `Choice → 1` :

```
12        *p=1;
(gdb) bt
#0  0x00005555555551fd in null_deref () at all_crashes.c:12
#1  0x0000555555555350 in main () at all_crashes.c:72
(gdb)
```

### Inspect Variables :

```
print p
```

OR

```
info locals
```

## Testing Each Crash Type in GDB :

### `Case 1 — NULL :`

```
run → 1 → bt
```

**Output :** `p = 0x0`

### `Case 2 — Wild Pointer :`

```
run → 2 → bt
```

**Output :** `p = random address (garbage)`

`Case 3 — Heap Overflow :`

```
run → 3
```

- Heap corruption happens silently.
- Might not crash immediately.

`Case 4 — Use After Free:`

```
run → 4
```

## Accessing freed memory.

`Case 5 — Stack Overflow :`

```
run → 5
```

Then :

```
bt
```

**Output :**

```
(gdb) bt
#0  0x0000555555555292 in stack_overflow () at all_crashes.c:56
#1  0x0000555555555297 in stack_overflow () at all_crashes.c:56
#2  0x0000555555555297 in stack_overflow () at all_crashes.c:56
#3  0x0000555555555297 in stack_overflow () at all_crashes.c:56
#4  0x0000555555555297 in stack_overflow () at all_crashes.c:56
#5  0x0000555555555297 in stack_overflow () at all_crashes.c:56
#6  0x0000555555555297 in stack_overflow () at all_crashes.c:56
#7  0x0000555555555297 in stack_overflow () at all_crashes.c:56
#8  0x0000555555555297 in stack_overflow () at all_crashes.c:56
#9  0x0000555555555297 in stack_overflow () at all_crashes.c:56
#10 0x0000555555555297 in stack_overflow () at all_crashes.c:56
#11 0x0000555555555297 in stack_overflow () at all_crashes.c:56
#12 0x0000555555555297 in stack_overflow () at all_crashes.c:56
#13 0x0000555555555297 in stack_overflow () at all_crashes.c:56
#14 0x0000555555555297 in stack_overflow () at all_crashes.c:56
#15 0x0000555555555297 in stack_overflow () at all_crashes.c:56
#16 0x0000555555555297 in stack_overflow () at all_crashes.c:56
#17 0x0000555555555297 in stack_overflow () at all_crashes.c:56
#18 0x0000555555555297 in stack_overflow () at all_crashes.c:56
#19 0x0000555555555297 in stack_overflow () at all_crashes.c:56
#20 0x0000555555555297 in stack_overflow () at all_crashes.c:56
#21 0x0000555555555297 in stack_overflow () at all_crashes.c:56
#22 0x0000555555555297 in stack_overflow () at all_crashes.c:56
#23 0x0000555555555297 in stack_overflow () at all_crashes.c:56
#24 0x0000555555555297 in stack_overflow () at all_crashes.c:56
#25 0x0000555555555297 in stack_overflow () at all_crashes.c:56
#26 0x0000555555555297 in stack_overflow () at all_crashes.c:56
#27 0x0000555555555297 in stack_overflow () at all_crashes.c:56
#28 0x0000555555555297 in stack_overflow () at all_crashes.c:56
#29 0x0000555555555297 in stack_overflow () at all_crashes.c:56
#30 0x0000555555555297 in stack_overflow () at all_crashes.c:56
#31 0x0000555555555297 in stack_overflow () at all_crashes.c:56
#32 0x0000555555555297 in stack_overflow () at all_crashes.c:56
#33 0x0000555555555297 in stack_overflow () at all_crashes.c:56
#34 0x0000555555555297 in stack_overflow () at all_crashes.c:56
#35 0x0000555555555297 in stack_overflow () at all_crashes.c:56
#36 0x0000555555555297 in stack_overflow () at all_crashes.c:56
#37 0x0000555555555297 in stack_overflow () at all_crashes.c:56
```

RECURSION CRASH LOOKS LIKE THIS.